

IHK – BubbleSort: Fragen & Antworten

Prüfungsjahre: 2017 – 2025 (Winter & Sommer, verschiedene Varianten)

Pseudocode-Regeln (IHK-Stil): **Funktion ... : Typ**, Zuweisung mit **=**, Schleifen: **für i = ... bis ...**, **solange**, **wenn ... dann**, **sonst**, **gib ... zurück**.

F1 – Klassischer BubbleSort (Ganzzahlen, aufsteigend)

Gekommen: Winter 2017/18, Sommer 2018, Winter 2020

Aufgabe (IHK-Stil): Gegeben ist `werte: Array<Integer>`. Entwickeln Sie einen Algorithmus, der das Feld aufsteigend sortiert. Geben Sie den Pseudocode an.

Lösung (Pseudocode)

```
Funktion bubbleSort(werte: Array<Integer>) : void
    für i = 0 bis länge(werte) - 2
        für j = 0 bis länge(werte) - 2 - i
            wenn werte[j] > werte[j + 1] dann
                temp = werte[j]
                werte[j] = werte[j + 1]
                werte[j + 1] = temp
            ende wenn
        ende für
    ende für
Ende Funktion
```

Antwort (Java-ähnlich)

```
void bubbleSort(int[] a){
    for (int i = 0; i < a.length - 1; i++){
        for (int j = 0; j < a.length - 1 - i; j++){
            if (a[j] > a[j + 1]){
                int t = a[j];
                a[j] = a[j + 1];
                a[j + 1] = t;
            }
        }
    }
}
```

F2 – Optimierter BubbleSort mit Abbruch (Flag)

Gekommen: Winter 2020, Sommer 2023

Aufgabe: Ergänzen Sie BubbleSort so, dass der Algorithmus stoppt, sobald in einer Runde kein Tausch erfolgt.

Lösung (Pseudocode)

```
Funktion bubbleSortOptimiert(werte: Array<Integer>) : void
    getauscht = true
    solange getauscht
        getauscht = false
        für i = 0 bis länge(werte) - 2
            wenn werte[i] > werte[i + 1] dann
                temp = werte[i]
                werte[i] = werte[i + 1]
                werte[i + 1] = temp
                getauscht = true
            ende wenn
        ende für
    ende solange
Ende Funktion
```

Antwort (Java-ähnlich)

```
void bubbleSortOptimiert(int[] a){
    boolean swapped = true;
    while (swapped){
        swapped = false;
        for (int i = 0; i < a.length - 1; i++){
            if (a[i] > a[i + 1]){
                int t = a[i];
                a[i] = a[i + 1];
                a[i + 1] = t;
                swapped = true;
            }
        }
    }
}
```

Typische Aufgabenformulierung: „Der Algorithmus soll verbessert werden, sodass er bei bereits sortierten Daten früher stoppt.“

F3 – Strings alphabetisch sortieren (lexikographisch)

Gekommen: Sommer 2021 (Vergleich nach Namen)

Aufgabe: Sortieren Sie `namen: Array<String>` alphabetisch aufsteigend.

Lösung (Pseudocode)

```
Funktion bubbleSortStrings(namen: Array<String>) : void
    für i = 0 bis länge(namen) - 2
        für j = 0 bis länge(namen) - 2 - i
            wenn namen[j] > namen[j + 1] dann    // lexikographischer
Vergleich
                temp = namen[j]
                namen[j] = namen[j + 1]
                namen[j + 1] = temp
            ende wenn
        ende für
    ende für
Ende Funktion
```

Antwort (Java-ähnlich)

```
void bubbleSortStrings(String[] s){
    for (int i = 0; i < s.length - 1; i++){
        for (int j = 0; j < s.length - 1 - i; j++){
            if (s[j].compareTo(s[j + 1]) > 0){
                String t = s[j];
                s[j] = s[j + 1];
                s[j + 1] = t;
            }
        }
    }
}
```

F4 – Sortieren mit Vergleichsfunktion (Comparator-Stil)

Gekommen: Sommer 2022 (Objekte mit Attribut/Comparator)

Aufgabe: Gegeben ist `kurse: Array<Tageskurs>` sowie `vergleiche(a,b): Integer` (>0 bedeutet $a > b$). Implementieren Sie die Sortierung aufsteigend.

Lösung (Pseudocode)

```
Funktion sort(kurse: Array<Tageskurs>, vergleiche: Funktion) : void
    für i = 0 bis länge(kurse) - 2
        für j = 0 bis länge(kurse) - 2 - i
            wenn vergleiche(kurse[j], kurse[j + 1]) > 0 dann
                t = kurse[j]
                kurse[j] = kurse[j + 1]
                kurse[j + 1] = t
            ende wenn
        ende für
    ende für
Ende Funktion
```

Antwort (Java-ähnlich)

```
void sort(Tageskurs[] k, java.util.Comparator<Tageskurs> cmp) {
    for (int i = 0; i < k.length - 1; i++) {
        for (int j = 0; j < k.length - 1 - i; j++) {
            if (cmp.compare(k[j], k[j + 1]) > 0) {
                Tageskurs t = k[j];
                k[j] = k[j + 1];
                k[j + 1] = t;
            }
        }
    }
}
```

F5 – Fehleranalyse (typische IHK-Frage)

Gekommen: Winter 2022, Sommer 2024

Aufgabe: Im folgenden Pseudocode steckt ein Schleifenfehler. Nennen und korrigieren Sie ihn.

Fehlerhafter Code

```
Funktion bubbleSort(a: Array<Integer>) : void
    für i = 0 bis länge(a) - 1           // Fehler: -1 ist zu weit
        für j = 0 bis länge(a) - 1 - i
            wenn a[j] > a[j + 1] dann
                t = a[j]; a[j] = a[j + 1]; a[j + 1] = t
            ende wenn
        ende für
    ende für
Ende Funktion
```

Begründung & Korrektur

Die Referenz `a[j + 1]` würde im letzten Durchlauf außerhalb des Bereichs liegen. Richtig ist:

```
für i = 0 bis länge(a) - 2
    für j = 0 bis länge(a) - 2 - i
```

F6 – Schritt-Simulation (Reihenfolge nach zwei Runden)

Gekommen: Sommer 2019, Winter 2021

Aufgabe: Simulieren Sie BubbleSort für `[7, 3, 4, 2]` nach zwei äußeren Runden.

Lösungsskizze

```
Start:           [7, 3, 4, 2]
(7,3) →          [3, 7, 4, 2]
(7,4) →          [3, 4, 7, 2]
(7,2) →          [3, 4, 2, 7]    // Ende Runde 1: 7 am Ende
(3,4) →          [3, 4, 2, 7]
(4,2) →          [3, 2, 4, 7]    // Ende Runde 2
```

Nach jeder Runde steht das größte Element des unsortierten Bereichs am Ende.

F7 – Theorie-Check (Kurzfragen wie in IHK)

1. Worst-Case-Laufzeit: **$O(n^2)$**
2. Ist BubbleSort stabil? → **Ja**
3. Früher Abbruch durch? → **Runde ohne Vertauschung**
4. Warum endet die innere Schleife bei **$n - 2 - i$** ? → Weil am Ende jeder Runde das größte Element bereits fixiert ist.

Merksatz: BubbleSort ist leicht zu schreiben und zu prüfen, aber für große Datenmengen langsam. Nach jeder Runde „blubbert“ das größte Element nach rechts.